SENSOR PROCESSING

21M.370

Digital Instrument Design

Ian Hattwick

Upcoming schedule

- Monday:
    - Readings
    - Micro Lecture
    - Performance?
- Wednesday (voxel)
    - Andi Otto
    - Grad presentations
    - Lab 5 performances
    - Lab 5 assemble MPR121
- Spring Break!
- Monday Mar 28 (online):
- Wednesday Mar 30 (voxel):

References

All references in Canvas unless a link is provided

https://hackaday.com/2015/12/09/embed-with-elliot-debounce-your-noisy-buttons-part-i/

Baalman, Marije. “From one range to another”. Just a question of mapping. 2021.

Sensor Signal Processing

How do we turn raw sensor data into signals that represent our actions?

How do we prevent false triggers or erroneous data?

What kinds of gestural data are useful?

Potentiometers and Buttons

- packages containing a sensor and a hardware interface
- designed with a clear gesture in mind
- physical form factor lends itself to easily mounting to an enclosure
- physical characteristics consider human factors

Discrete Signals

Either 0/1, up/down

Useful states:

- Down - transition from up to down
- Pressed - constant value of 1
- Up - transition from down to up
- Released - constant value of 0
  Need to remember previous state to compare

Discrete Signals

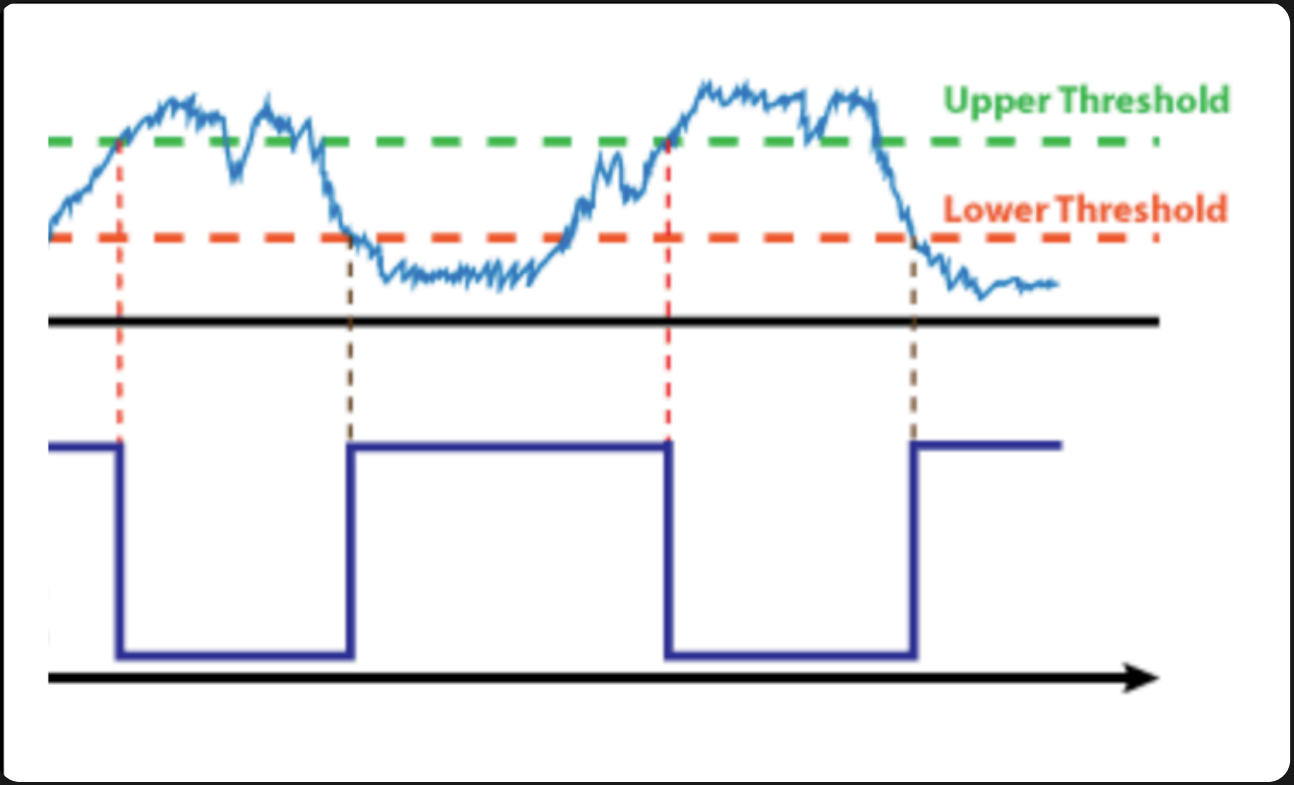
Possible errors:

- mechanical bounce: button contact ‘chatter’s when pressed, creating a series of on/off values
  Debounce techniques:
- forced delay: prevent retriggers until N ms
- lowpass filter & schmitt trigger
- pattern filtering

Schmitt Trigger

Schmitt trigger

- separate thresholds for up and down states
- very standard for discrete inputs

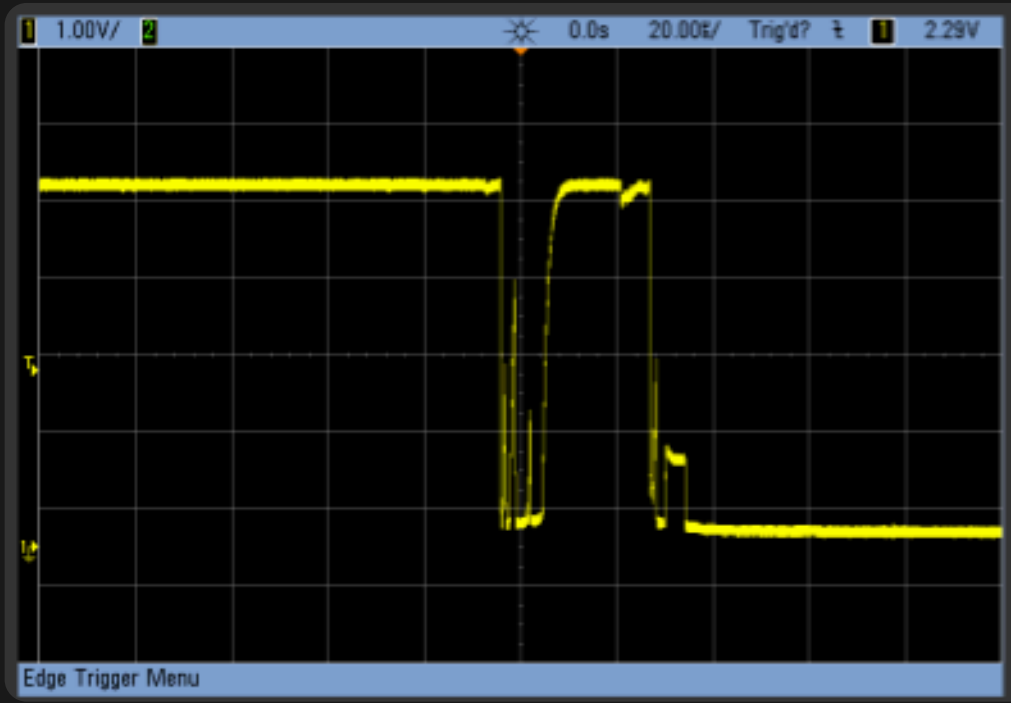


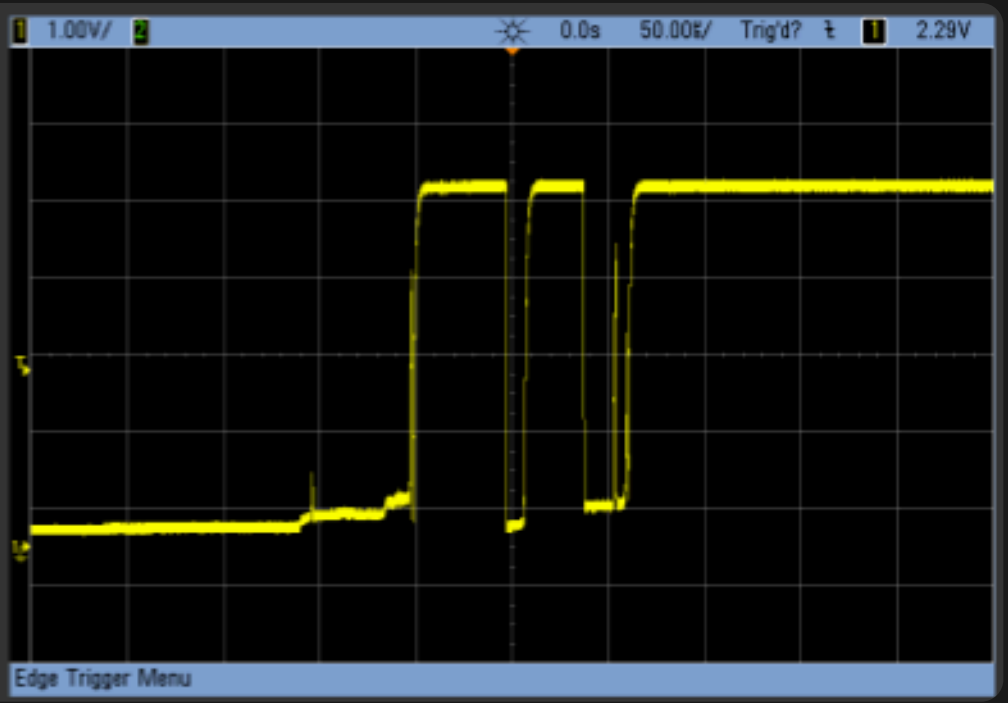
Pattern Matching

Pattern matching

- look for patterns over time to determine state
- create array of values [oldest, ... , newest]
- ideal patterns:
- [00000001] is change from low to high
- [11111110] is change from high to low

  Real world patterns
- [00000001] instantaneous change when button is pressed
- [10010100] possible pattern when releasing with bounces
- [10000000] identify release only at end of button bouncing


1.00V/
0.0s
20.00µ/
Trig'd?
2.29V
Edge Trigger Menu


1.00V/
0.0s
50.00µ/
Trig'd?
2.29V
Edge Trigger Menu

Discrete Signals

Issues:

- when is button state change identified?
- lowpass filters add latency
- pattern matching may add latency
    - latency on releases less important than presses? Other modalities
- double-click
- long/short press
- length of button press as signal

Continuous signals

Interest in range of values related to a gesture

Potentiometer:

- gesture of turning a knob related to output signal
- problems: incomplete range, noisy signal

Capacitive sensor

- range of values may constantly change
- no clear limit to range of values
- very noisy

Continuous signals

One-pole lowpass filter

`y[n] = a * x[n] + (1 - a ) * y[n-1], a=smoothing value`

- high values of *a* give less smoothing
- It uses a feedback loop, so the calculation needs to run continuously - e.g., it needs to get called every loop
- easy and effective for simple noise

```
def read(self):
    value = self.pot.value
    # onepole lowpass filter
    value = value*(1-self.onepole_alpha) + self.prev_value*self.onepole_alpha

    if abs(value - self.prev_value) >= 100:
        self.prev_value = value
        midi = int(value) >> 9
        if midi != self.prev_midi:
            self.prev_midi = midi
            return midi
```

Continuous signals

processing a window of values

- store an array of N values
- push new data to array and remove oldest element
- then process entire array

Moving average: take average of array

- similar to lowpass filter but more complicated/ takes more memory
- good for generally noisy signals

Median Filter

Median filter

- removes high and low values
- good at removing noise spikes

Ranges

Generic scaling:

```
def scale(input, inLow, inHigh, outLow, outHigh, curve = 1):
    val = (input-inLow)/(inHigh-inLow)
    val = pow(val,curve)
    val = val*(outHigh-outLow) + outLow
    return val
```

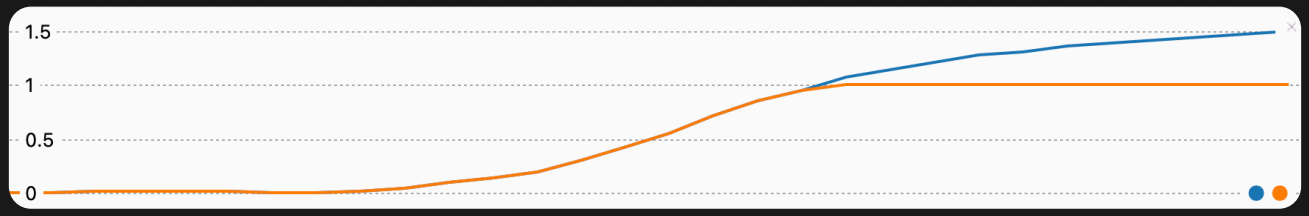
Doesn’t clip output
think of how curve would affect a range of 0-1

- outside of that range things can get screwy!

Clipping

What happens when a value goes out of range?

- gets limited to range
- or is allowed to exceed range
  Too much clipping can lead to a boring experience
- sometimes unexpected behaviour at extremes can be interesting!
  Design systems to tolerate wide values, even if they become unpredictable

Curve = 1

Curve = 2